
NOTE

When you instantiate an object from a class, Python takes care of constructing the object (allocating memory) for you. The special `__init__()` method is called the “initializer” method, where you give variables initial values. (Most other OOP languages require a method named `new()`, which is often referred to as a constructor.)

Scope and Instance Variables

In procedural programming, there are two principal levels of scope: variables created in the main code have *global* scope and are available anywhere in a program, while variables created inside a function have *local* scope and only live as long as the function runs. When the function exits, all local variables (variables with local scope) literally go away.

Object-oriented programming and classes introduce a third level of scope, typically called *object scope*, though sometimes referred to as *class scope* or more rarely as *instance scope*. They all mean the same thing: the scope consists of all the code inside the class definition.

Methods can have both local variables and *instance variables*. In a method, any variable whose name does not start with `self.` is a local variable and will go away when that method exits, meaning other methods within the class can no longer use that variable. *Instance variables* have object scope, which means they are available to *all* methods defined in a class. Instance variables and object scope are the keys to understanding how objects remember data.

instance variable

In a method, any variable whose name begins, by convention, with the prefix `self.` (for example, `self.x`). Instance variables have object scope.

Just like local and global variables, instance variables are created when they are first given a value and do not need any special declaration. The `__init__()` method is the logical place to initialize instance variables. Here